

ОС і СП

Лабораторна робота 11: інтерпретація формул без рангів операцій

Мета роботи. Вивчити часткові теоретичні методи проектування інтерпретаторів і наданий приклад інтерпретатора формул за операціями без рангів. Доповнити інтерпретатор іншими бінарними і унарними операціями. Реалізувати можливість вибору системи числення. Розуміння синтаксичних помилок. Розширити інтерпретатор можливостями контролю коректного зображення формул.

Зауважимо, що формула може бути елементом будь-якого текстового документа, і не лише текстового, і не лише математичного.

Матеріали до завдання

Теоретичний матеріал початкового рівня для цієї роботи викладений в лекції "L11 Обчислення формул інтерпретацією.pdf" в розділах "Правила будови синтаксичних визначень" і "Найпростіший приклад інтерпретації".

Для глибшого вивчення питань цього розділу курсу і для виконання лабораторних робіт 11 і 12 треба звертатись за покликаннями:

Вікіпедія. Інтерпретатор мови програмування:

<https://uk.wikipedia.org/wiki/Інтерпретатор>

Інтерпретатор:

<http://kytok.org.ua/?p=732>

Вікіпедія. Компілятор:

<https://uk.wikipedia.org/wiki/Компілятор>

Для *практичного проектування і програмування* надається файл `simple.py`, як базовий для подальшої роботи методами розширення, рефакторингу, перекодування тощо.

Завдання частина 1 (ознайомлення, вивчення)

1. На початку виконати різні обчислювальні експерименти з доданим файлом `simple.py`. Зокрема, в останньому розділі файлу для виконання

```
"if __name__ == "__main__" :
```

пробувати виконати різні підстановки в оператор `"formula = ..."`, щоб мати достатнє уявлення про методи розбору виразів та їх інтерпретацію на основі сканування і зображення лексемами.

Пояснення до п.1.

1.1. В одному рядку текстового файлу чи в деякому іншому текстовому рядку записана послідовність десяткових цілих чисел і знаків арифметичних операцій (формула) в *інфіксній* формі. Наприклад:

$$65 + 122 - 99 / 6 - 12 * 2 + 1$$

Необхідно виконати обчислення формули за такими *правилами і обмеженнями*: числа десяткові цілі; допустимі операції `+`, `-`, `*`, `/`; операції рангу не мають і їх виконують зліва направо в порядку запису; результатом кожної операції є ціле або дійсне число; формула записана коректно, зокрема, для запису чисел використані лише десяткові цифри і немає операції ділення на нуль. Така формула нагадує просту послідовність операцій на калькуляторі.

1.2. Програмний код мовою Python інтерпретатора таких формул є в доданому файлі `simple.py`. Вивчити програмний код. Можна прийняти його за основу або реалізувати іншою алгоритмічною мовою, використовуючи методи рефакторингу чи перекодування.

Інтерпретатор працює за два перегляди тексту формули. Два перегляди є оптимальними для спрощення алгоритмів інтерпретації. До початку першого перегляду метод `delblank()` викреслює з тексту всі незначущі пропуски. За першим переглядом будують список лексем цілої формули. За другим переглядом обчислюють формулу, якщо не було помилок за першим переглядом. У випадку помилок друкують найпершу помилкову літеру.

Оператори `print()` методу `calc()` можна розкоментувати, щоб побачити проміжні результати процедури обчислення. Можна також додати для експериментів інші оператори `print()` до тексту методів, і отримати спостереження за ходом виконання інтерпретатора.

1.3. Результати кроків виконання інтерпретації є такі:

задано	"65 + 122 - 99 / 6 - 12 * 2 + 1"
викреслити пропуски	"65+122-99/6-12*2+1"
поділити на лексеми	['65', '+', '122', '-', '99', '/', '6', '-', '12', '*', '2', '+', '1']
додати знак "+" на початок	['+', '65', '+', '122', '-', '99', '/', '6', '-', '12', '*', '2', '+', '1']
цикл обчислення пар [знак,операнд]	['+', '65'], ['+', '122'], ['-', '99'], ['/', '6'], ['-', '12'], ['*', '2'], ['+', '1']

2. Виконайте декілька експериментів обчислення різних формул, в тому числі формул з помилками.

Завдання частина 2 (практика проєктування і програмування)

Основна частина завдання викладена в пп.3,4,5,6. Щоб виконати ці пункти, треба на початку побудувати розширене синтаксичне означення правил запису формули відповідно до кожного пункту, взявши за основу синтаксичне означення формули з розділу "Найпростіший приклад інтерпретації" лекції L11. Після цього виконати редагування програмного коду інтерпретатора.

УВАГА! Рекомендуємо спочатку повністю реалізувати п.3 завдання, виконати повне тестування, отримати правильні результати, тоді перейти до п.4 завдання. Так само далі - до п.5 і до п.6.

3. Додайте до інтерпретатора `simple.py` інші бінарні операції, наприклад, "остача від ділення", "менше з останнього обчисленого і наступного". Визначити чотири інші бінарні операції крім реалізованих `+`, `-`, `*`, `/`. Бінарні операції не зобов'язані бути стандартними, можна обґрунтувати власну числову операцію з міркувань потенційного застосування.

4. Додайте до інтерпретатора 3 унарні операції (стандартні чи власні), наприклад, *корінь квадратний* чи *sin*. Унарну операцію в нашому випадку інтерпретації без рангів застосовують до останнього обчисленого результату (*постфіксна* форма). Наприклад, запис

$$25 + 52 \text{ sqrt} - 120 \text{ sin} * 2$$

означає

$$(\text{sin} (\text{sqrt}(25 + 52) - 120)) * 2$$

5. Реалізуйте можливість виконання обчислень в інших системах числення: *двійкова*; *шістнадцяткова*. Для цього треба зафіксувати допустимі літери для запису цифр і контролювати їх використання. Тоді можна, наприклад, виконувати переведення кожного числа з іншої системи числення в десяткову, самі обчислення можна виконати, як раніше, в десятковій системі числення, в кінці показати результат у вхідній системі числення.

Подумати, як позначити безпосередньо в формулі систему числення. Повний запис цілої формули разом з позначеннями має укладатись в один рядок.

5.1. Всі числа формули записані в однаковій системі числення. Результат – в тій самій системі числення.

5.2. Кожне число може мати запис своєю системою числення незалежно від решти чисел. Отже треба передбачити позначення системи числення кожного числа і це все має бути в лінійній формі. Результат – в якій системі? (Обґрунтуйте)

6. Контроль помилок запису формули.

Для виконання попередніх пунктів завдання формулу вважали записаною коректно. В цьому пункті треба доповнити інтерпретатор засобами контролю за випадками неправильного запису формул. Отже найперше треба скласти перелік *потенційних* порушень правил запису формул. За кожним потенційним порушенням додати до інтерпретатора програмні елементи контролю. Спосіб контролю покладаємо на власний алгоритм. Але повідомлення про помилку має бути в формі, зрозумілій користувачу нашого продукту – ергономіка!

В файлі інтерпретатора `simple.py` показано лише приклад виявлення простої помилки на рівні запису окремої лексеми. В цілому помилки можуть бути будь-якої форми невідповідності правилам запису формули, наприклад, дві операції записані підряд чи формат запису числа не відповідає системі числення.

Користуючись нагодою, надаємо покликання для ширшого вивчення проблем помилок в програмуванні:

Що таке синтаксична помилка і як її вирішити?

<https://www.geeksforgeeks.org/what-is-a-syntax-error-and-how-to-solve-it/>

Вікіпедія. Синтаксис (мови програмування):

[https://en.wikipedia.org/wiki/Syntax_\(programming_languages\)](https://en.wikipedia.org/wiki/Syntax_(programming_languages))

Вікіпедія. Синтаксична помилка:

https://en.wikipedia.org/wiki/Syntax_error

Що таке синтаксис? Компоненти, правила та поширені помилки:

<https://www.geeksforgeeks.org/what-is-syntax-components-rules-and-common-mistakes/>

Звіт за роботу

В результаті виконання завдання надіслати:

1) побудовані власні розширені синтаксичні означення формул для кожного пп.3,4,5 завдання окремо;

2-3) програмний код файлу python доповненого інтерпретатора `simple.py`, спільний для всіх пп.3,4,5,6 завдання;

2-3) якщо реалізація іншою мовою, ніж python, тоді надіслати архів проєкту;

4) в будь-якому варіанті реалізації програмний код має бути коментований в тих позиціях, де були внесені зміни, щоб бачити різницю від базового варіанту `simple.py`;

5) тестові приклади формул правильного запису, на яких виконали перевірку інтерпретатора, разом з отриманими результатами; звертаємо увагу, що перелік тестів має бути достатнім для повної демонстрації реалізації всіх елементів завдання – це суттєво впливає на оцінку; вибір тесту і запуск на виконання мають бути простими;

6) тестові приклади формул, в записі яких є помилки; пояснити одним реченням зміст помилки; так само вибір тесту і запуск на виконання мають бути простими;

7) відповіді оформити коротким звітом – окремим текстовим файлом.

Всі файли надіслати *окремими вкладеннями*, НЕ АРХІВОВАНІ, крім можливо, архіву програмної реалізації.